ChessQ Platform Spec v2.1 (Adapted)

ChessQ Platform — Complete System Specification

Document Version: 2.1 (Adapted Edition) **Date:** 2026-07-02 **Original Authors:** Steven Songwe and Enock Chizuzu **Adapted by:** Enock Chizuzu, with Claude (PM/Advisor) — to align tooling and phasing with current project state **Classification:** Confidential — For Internal Development Use Only

> **What changed from v2.0 and why:** This edition keeps the same product vision, features, and long-term direction as the original document. Two things changed: (1) the infrastructure/tooling — swapped from a Python/Redis/Kubernetes microservices stack to Next.js + Supabase, which one or two developers can actually build and operate; (2) the development roadmap — tightened into shorter phases matching our real team size (effectively one developer coding, one contributing chess-domain expertise) and current progress (homepage UI complete, nothing else built yet). Everything else — the “what,” not the “how” — is unchanged. Sections un-related to backend architecture (Frontend Pages & Features, Product Vision) are preserved almost verbatim since they don’t depend on the underlying stack.

Table of Contents

---

1. Product Vision

1.1 Mission Statement

Provide a robust, real-time chess playing experience that can be used as a standalone platform, with a path toward integrating with a broader chess ecosystem later.

The platform handles user pairing, move validation, time controls, game state persistence, fair play, tournaments, learning tools, and a social layer — built incrementally, starting with the smallest version that is genuinely playable.

1.2 Core Principles

- **Correctness first:** Every move validated against official rules (castling, en passant, promotion, check/mate/stalemate, draws) — enforced via the `chess.js` library, not hand-written logic.
- **Low latency:** Real-time moves via Supabase Realtime (built on Postgres logical replication + WebSockets under the hood), targeting sub-200ms median delivery between players.
- **Managed infrastructure over self-hosted orchestration:** No Kubernetes, no Redis cluster to operate. Supabase (Postgres + Auth + Realtime) and Vercel (hosting) handle the operational burden so a small team can focus on product.
- **Mobile-first UX:** All pages designed for mobile first, then adapted to desktop.
- **Isolated but integrable:** The platform hosts games and provides basic learning; it can push completed game data to external services later (e.g., an AI coach), same as originally envisioned.

1.3 Product Goals — Full Vision (Unchanged)

These remain the long-term destination. Not all are V1 — see Section 19 for what's actually built first.

- User registration / authentication
- Lobby, quick pairing (random), custom challenges
- Multiple time controls (bullet, blitz, rapid, classical)
- Real-time board synchronization, clocks, full rules enforcement
- Tournament system (Arena & Swiss)
- Leaderboard (global, per time control)
- Basic learning tools (daily puzzle, opening explorer, simple lessons)

- Game history, replay, PGN export
- Rating system (Glicko-2, or Elo as a simpler interim)
- Spectator mode
- Notifications & private messaging

1.4 Success Metrics

Kept directionally the same, adjusted to be realistic for early stage:

- Move delivery latency: aim for well under 500ms in early testing; tighten toward 200ms as the platform matures.
- Uptime: not a formal SLA at V1 — Vercel/Supabase managed uptime is the baseline.
- Game completion rate: no corrupted game state — this is a hard requirement from day one regardless of scale.
- Early-stage metric that matters more than the above: **did a real human complete a real game and want to play again.**

---

2. System Architecture

2.1 High-Level Diagram (Revised)

Key decisions (revised):

- No separate microservices, no message broker to run — Supabase is a managed backend-as-a-service covering auth, database, and realtime pub/sub in one product.
- Game logic (move validation, rules) runs via `chess.js` — either client-side with server-side re-validation in a Next.js API route, or fully server-authoritative in an API route, depending on how much trust we place in the client as the platform matures. **V1 approach: server-authoritative validation in a Next.js API route before any move is written to the database** — this preserves the original spec's "server is the authority" principle (Section 17) without needing a standalone stateful game service.
- Postgres row changes (e.g., a new move inserted into a `games` table) are pushed to subscribed clients via Supabase Realtime — this replaces the original Redis Pub/Sub role.
- Horizontal scaling, sharding, and container orchestration are explicitly **deferred** (see Section 16) until there's an actual scale problem to justify the complexity.

---

3. Core Backend Components (Revised)

The original spec split responsibilities into nine separate services (Auth, Lobby, Game, Recorder, Rating, Tournament, Notification, Learn). At current team size, these remain **logical responsibilities**, not separate deployed services — they live as modules within the Next.js app and Supabase, and can be physically separated later if/when scale demands it.

3.1 Auth Module

- User registration, login — handled by **Supabase Auth** (email/password to start; OAuth providers like Google can be added later with minimal effort).
- Session/token handling is managed by Supabase's client libraries — no custom JWT issuance code to write or maintain.

3.2 Lobby / Matchmaking Module

- Manage challenges (create custom, list open) and track online users.
- V1: link-based challenges ("share this link to invite someone") rather than a full matchmaking pool — see Section 5.

3.3 Game Module (the core of V1)

- Move validation via `chess.js` (server-side, authoritative).
- Game state (FEN, move history, clocks) persisted in Postgres via Supabase.
- Real-time sync via Supabase Realtime subscriptions on the `games` table.

3.4 Game Recorder

- On game end, final PGN and result are already in the `games` row — no separate archiving service needed; it's the same row, just marked complete.

3.5 Rating Module

- V1: simple Elo, calculated in application code after each rated game, written directly to the `users` table.
- Glicko-2 (as originally specified) is a **Later** upgrade — same data shape, more sophisticated math, not urgent for early users.

3.6 Tournament Module

- **Deferred** — see Section 19. Not part of V1.

3.7 Notification Module

- V1: minimal, in-app only (e.g., "your opponent moved") via Realtime subscriptions.
- Email/push notifications: **Later**.

3.8 Learn Module

- **Deferred** — puzzles, lessons, opening explorer, endgame practice all come after core play is validated.

---

4. Real-Time Communication (Revised)

4.1 Protocol: Supabase Realtime (Postgres Changes)

Instead of hand-rolling a WebSocket protocol with custom JSON frames, we subscribe to changes on relevant

Postgres tables. Conceptually equivalent to the original spec's message types, but the transport is provided, not built:

Conceptual event mapping (same intent as original `game.move`, `game.over`, etc.):

Original concept	Revised implementation
`move` (client → server)	POST to a Next.js API route (`/api/games/[id]/move`) — server validates via chess.js, writes new FEN + move to DB
`position_update` (server → client)	Supabase Realtime subscription fires automatically when the `games` row updates — client re-renders board from new FEN
`game_over`	Same row update, with a `status` / `result` column change, detected via the same subscription
`resign` / `draw_offer`	Small dedicated API routes, writing a status change to the `games` row
`chat`	Optional, deferred — see Section 15

4.2 Connection Lifecycle (Revised)

1. Client authenticates via Supabase Auth, receives a session.
2. Client opens a Realtime subscription scoped to a specific game ID (e.g., `games:id=eq.<uuid>`).
3. Moves are submitted via a normal HTTPS POST to an API route (simpler and easier to debug than raw WebSocket frames for a solo/near-solo dev), not sent directly over the realtime channel.
4. Reconnection: since the source of truth is a Postgres row (not in-memory state on a stateful server), a disconnected client can always re-fetch the current game state on reconnect — no custom "grace period" server logic required for V1. (A UX-level grace period, e.g. before declaring someone abandoned, can be added later.)

4.3 Clock Synchronization

- V1: clock state stored as a timestamp + remaining-time pair on the `games` row, recalculated on each render from server time — avoids needing a constantly-ticking server process.
- Precision suitable for rapid/classical time controls initially; bullet-level (sub-second) precision is a **Later** refinement once the core loop works.

---

5. Matchmaking Engine (Revised — Simplified for V1)

5.1 V1: Link-Based Challenges (not a matchmaking pool)

1. Player creates a game, gets a shareable link/code.
2. Second player opens the link, joins the game.
3. No rating-based pairing pool, no Redis sorted sets, no matchmaking worker.

This intentionally defers the original Section 5's rating-range pairing pool (`±200 rating`, 30s timeout, continuous worker) — that's real infrastructure worth building once there's a large enough pool of concurrent users for random pairing to make sense. Before then, it's complexity with no payoff.

5.2 Custom Challenges

- Same concept as original: create a challenge (opponent, time control, rated/casual), opponent notified,

acceptance creates a game. Implemented as a Next.js API route + DB row instead of a dedicated Lobby Service.

5.3 Later: Random/Rating-Based Matchmaking

Revisit once there are enough concurrent users that “share a link” is a worse experience than “get paired automatically.”

---

6. Game Engine & Board Mechanics

6.1 Board Representation

- **`chess.js`** (JavaScript/TypeScript) replaces the original `python-chess` — same category of library (rules engine), different language to match our all-TypeScript stack.
- Client renders the board using **`react-chessboard`**, driven by FEN, matching the original spec’s approach exactly (Section 6.1 already specified `react-chessboard` for the client — that part of the plan carries over unchanged).

6.2 Move Execution Flow (unchanged in principle)

1. Player drags/selects a piece client-side; move sent to server as UCI or SAN.
2. Server validates via `chess.js`’s move-legality check.
3. If illegal, return an error response.
4. If legal, apply the move, update the stored FEN/PGN and clock, check for game end.
5. Realtime subscription pushes the update to both players (and spectators, once that feature exists).
6. On game end, update the `games` row’s status — no separate event publish needed since it’s the same persisted record.

6.3 Special Moves & UI — unchanged

Castling, en passant, promotion, draw offers, and resignation are handled identically to the original spec; `chess.js` supports all of these natively, same as `python-chess` did.

6.4 Draw Detection — unchanged

`chess.js` provides equivalent methods (`isThreefoldRepetition()`, `isDraw()`, `isStalemate()`, `isInsufficientMaterial()`) to the originally specified `python-chess` calls. V1 uses auto-declaration, same as original.

6.5 Clocks — see Section 4.3

6.6 Disconnection Handling (simplified for V1)

Since game state lives in Postgres (not server memory), a disconnected player’s game isn’t at risk of being lost — they can reconnect and refetch state at any time. A stricter “clock keeps running, timeout = loss” rule (as in the original spec) is straightforward to add once basic reconnection is proven to work.

---

7. Move Validation & Rules Enforcement

- Legality: `chess.js`’s built-in move validation — checks turn, path obstruction, and whether the move leaves the mover in check. Same guarantee as the original `python-chess` approach.
- **Server is always the authority** — client-side move rendering is provisional until the server confirms it, exactly matching the original spec’s Section 17 security principle.

- Anti-cheat rate limiting and timestamp recording: see Section 9 (unchanged intent, deferred implementation).

---

8. Time Controls

Unchanged from the original — presets, custom time control validation, and storage format (`"base+increment"` string) all carry over as-is; this logic is stack-independent.

Category	Popular Settings
Bullet	1+0, 2+1, 1+2
Blitz	3+0, 3+2, 5+0, 5+3
Rapid	10+0, 10+5, 15+10, 30+0
Classical	30+20, 60+0, 90+30

V1 recommendation: launch with **one or two time controls** (e.g., 10+0 and 5+0) rather than the full matrix — reduces testing surface while validating the core play loop.

---

9. Anti-Cheat Considerations

Same V1-minimum posture as the original document: no real-time engine detection at launch. Move-rate limiting and full move/timestamp logging (for later analysis) remain good, cheap practices to build in from day one, since it's easy to add now and hard to retrofit. Deeper anti-cheat (behavioral analysis, engine-correlation detection) stays firmly in **Later**, and only becomes a real priority once games involve stakes (ranked ratings, paid entry).

---

10. Rating System (Revised for V1)

- **V1: simple Elo**, not Glicko-2. Same underlying idea (numeric skill rating that updates after each game) but far simpler to implement correctly as a first version — fewer parameters (no rating deviation, no volatility), well-documented formula, less surface area for bugs in a system that directly affects user trust.
- Glicko-2, provisional ratings, and per-time-control rating tracks (all in the original Section 10) remain the **Later** upgrade path — the data model can accommodate the switch without a rewrite, since it's still "a number per user per time control" underneath.

---

11. Database Design (Revised — Trimmed to V1 Tables)

All tables live in Supabase Postgres. Schema below covers what's needed for V1 (auth, users, games, moves, challenges). Tournament, puzzle, lesson, notification, and messaging tables from the original spec are preserved in Section 19's "Later" reference but not created until those features are actually built — an empty table that might be wrong later is worse than no table yet.

11.1 Users (extends Supabase's built-in `auth.users`)

```
create table public.profiles (
  id uuid references auth.users(id) primary key,
  username text unique not null,
```

```
  display_name text,
  rating integer default 1200,
  created_at timestamptz default now()
);
```

11.2 Games

```
create table public.games (
  id uuid primary key default gen_random_uuid(),
  white_id uuid references public.profiles(id),
  black_id uuid references public.profiles(id),
  time_control text,
  rated boolean default false,
  fen text not null,
  pgn text,
  status text default 'active', -- active, checkmate, stalemate, draw, resigned, timeout
  result text, -- white, black, draw, null if ongoing
  created_at timestamptz default now(),
  ended_at timestamptz
);
```

11.3 Moves (optional for V1, useful for replay)

```
create table public.game_moves (
  game_id uuid references public.games(id),
  move_number integer,
  uci text,
  san text,
  created_at timestamptz default now(),
  primary key (game_id, move_number)
);
```

11.4 Challenges

```
create table public.challenges (
  id uuid primary key default gen_random_uuid(),
  challenger_id uuid references public.profiles(id),
  challenged_id uuid references public.profiles(id),
  time_control text,
  rated boolean default false,
  status text default 'pending', -- pending, accepted, declined, expired
  created_at timestamptz default now()
);
```

(Tournaments, puzzles, lessons, notifications, messages, rating history tables from the original v2.0 spec are retained as reference in the full vision document but intentionally not created yet — see Section 19.)

---

12. API Design (Revised)

12.1 REST-style Endpoints (Next.js API Routes, base `/api`)

Auth — mostly handled by Supabase client SDK directly; minimal custom routes needed.

Games - `POST /api/games` — create a new game (returns shareable link/code) - `POST /api/games/[id]/join` — second player joins - `POST /api/games/[id]/move` — submit a move (server validates via chess.js) - `POST /api/games/[id]/resign` - `POST /api/games/[id]/draw-offer` - `GET /api/games/[id]` — fetch current state (for reconnect/refresh)

Challenges - `POST /api/challenges` - `POST /api/challenges/[id]/accept`

(Tournaments, leaderboard, learn, notifications, messages endpoints: deferred, listed in the original spec as future reference.)

12.2 Realtime

No separate WebSocket endpoint to build — Supabase Realtime subscriptions are configured client-side against specific tables/rows, replacing the original `ws://platform.example/ws` custom endpoint entirely.

---

13. Background Jobs & Services (Revised)

Most of the original background jobs (Matchmaking Worker, Game Archiver, Rating Updater, Tournament Pairing Scheduler, Dead Game Cleaner, Daily Puzzle Scheduler, Leaderboard Cache, Notification Dispatcher) either aren't needed yet (tournaments, puzzles) or collapse into simple logic that runs at the moment it's needed rather than as a standing background process:

- **Rating update:** happens synchronously in the move/game-end API route, not a separate subscriber process.
- **Dead game cleanup:** deferred — not urgent at V1 scale; can be a scheduled Supabase Edge Function later if abandoned games become a real problem.
- Everything else in this section: **Later**, once the corresponding feature (tournaments, puzzles, leaderboard caching at scale) is actually built.

---

14. Frontend Pages & Features

(This section is preserved close to verbatim from the original — it doesn't depend on backend architecture, and is a good reference for the full-vision page set. Bold notes indicate V1 vs. Later status.)

14.1 Homepage — ✅ V1, largely built

Quick Play buttons, Active Games, Tournament Spotlight, Recent Activity, mini leaderboard. Currently implemented with mock data; will be wired to real data as backend features land.

14.2 Play Area — V1, next to build

Board (drag-and-drop, legal move highlights, last-move highlight), digital clocks, move list, resign/draw buttons, post-game result overlay, board orientation flip for black. Pre-move support: **Later**.

14.3 Tournaments — Later

Tournament lobby, detail pages, live standings, spectating, Arena & Swiss modes. Full feature set unchanged from original spec — just not V1.

14.4 Leaderboard — Later

Per-time-control tabs, ranked list, auto-refresh. Simple version could come early once Elo ratings exist; full version later.

14.5 Learn — Later

Daily puzzle, opening explorer, lessons, endgame practice. Unchanged concept, deferred build.

14.6 Profile — V1-lite, then expand

V1: avatar, username, current rating, recent games list. Rating graph, achievements, friends list: **Later**.

14.7 Settings — V1-lite, then expand

V1: account basics (email/password via Supabase Auth UI), board theme. Full preference set (notifications, privacy, auto-queen, etc.): **Later**.

---

15. Notifications & Messages

- **V1:** minimal in-app notification only for "it's your move" / "game ended," via Realtime subscription — no dedicated notification service or table required yet.
- Private messaging, friend system, per-type notification preferences: **Later**, unchanged in concept from the original spec's Sections 15.1–15.3.

---

16. Scalability & Deployment (Revised)

This is the section with the largest change from the original, and worth being direct about:

- **No Kubernetes.** Vercel handles deployment, scaling, and global CDN distribution for the frontend automatically.
- **No Redis, no message broker, no manual sharding.** Supabase manages Postgres scaling, connection pooling, and realtime fan-out.
- **No sticky sessions to configure** — there's no stateful in-memory game server to keep affinity with; game state lives in the database, which any request can read.
- This tradeoff means lower theoretical ceiling than a custom microservices architecture — but a ceiling that's far above what a pre-revenue platform will hit, and one we can migrate away from later, with real usage data, if it ever becomes the actual bottleneck. Optimizing for scale we don't have yet is exactly the kind of overbuilding this revision is meant to prevent.
- Global multi-region distribution (original Section 16's "future" note): stays a future consideration, unchanged.

---

17. Security Requirements

Largely unchanged in principle, simplified in implementation:

- TLS: handled automatically by Vercel and Supabase.
- Auth tokens: managed by Supabase Auth (secure session handling out of the box) instead of hand-rolled JWT issuance/rotation.
- Rate limiting: apply at the Next.js API route level for move submission and other write endpoints.
- Input validation: use a schema validation library (e.g., `zod`) in API routes; Supabase's client uses parameterized queries, mitigating SQL injection risk by default.
- Password hashing: handled by Supabase Auth internally — no custom bcrypt implementation needed.
- **Game state integrity: server is the authority — never trust client.** This principle from the original spec is unchanged and non-negotiable regardless of stack.

---

18. Integration Points with External Systems

Unchanged from the original in concept:

- **AI Chess Coach integration:** once a game ends, its PGN is available in the `games` table; a future

integration could call an external analysis service with that PGN, or expose a webhook/API for it, same idea as the original Redis-event-based design — just triggered differently (e.g., a Supabase Database Webhook on game completion, instead of a custom Redis publish).
- **External puzzle/learn content:** same as original — future consideration.
- **Third-party auth (Google, Apple):** Supabase Auth supports OAuth providers natively — this is actually *easier* to add than in the original spec's custom Auth Service.

---

19. Development Roadmap (Revised)

This is the other major change from the original document. The original Section 19 phased things in **months**, assuming a funded team. Revised for our actual situation: effectively one developer coding, one contributing chess-domain expertise, ~20 hrs/week combined, currently at "homepage UI complete, nothing else built."

Phase	Focus	Key Deliverables
0 — Now (Week 1)	Local playable board	`chess.js` + `react-chessboard` integrated, hotseat two-player mode (no networking), rules fully enforced
1 — Weeks 2-3	Backend foundation + live play	Supabase set up (auth, DB, realtime); link-based two-player live games; basic clocks; game state persisted
2 — Week 4	Real user pilot	5-10 real people play real games; fix top issues found
3 — Month 2	Depth on core play	Elo ratings, game history/replay, PGN export, reconnect handling, Stockfish-powered post-game analysis (basic)
4 — Month 3+	Social & competitive layer	Custom challenges polish, simple leaderboard, basic profile expansion. Tournaments (Arena first, Swiss later) — only once organic demand is visible
5 — Later	Learning & content	Daily puzzles, opening explorer, lessons, endgame practice
6 — Later	Polish & scale	Glicko-2 upgrade, anti-cheat hardening, admin panel, friend system, messaging, notifications expansion, migrate off Supabase-managed simplicity only if real scale data justifies it
7 — Beyond	Ecosystem	AI Coach integration, mobile apps, club/organizer tools, advanced analytics

Explicitly not started until their phase arrives: Kubernetes, Redis, tournaments, anti-cheat beyond basic rate-limiting, Glicko-2, payments. This list matters as much as what's in Phase 0-2 — it's the scope boundary.

20. Glossary

Unchanged from the original:

Term	Definition
UCI	Universal Chess Interface — move notation like e2e4
FEN	Forsyth-Edwards Notation — describes a board position
Glicko-2	Advanced rating system with rating deviation and volatility
Elo	Simpler, widely-used rating system; V1 rating choice for ChessQ
Swiss	Tournament format with fixed rounds, pairing by score
Arena	Continuous tournament where players join/leave freely
Realtime (Supabase)	Managed pub/sub over Postgres row changes, replacing custom WebSocket/Redis infrastructure in this revision
PGN	Portable Game Notation — standard chess game record format

---

Document Endorsement

This adapted specification preserves the full product vision and feature set of the original v2.0 document while aligning technical implementation and phasing to the team's actual current capacity and progress. It should be reviewed and agreed by both authors before being handed to any development tool (including AI coding assistants) as a build plan. As with the original, all future changes should be reflected in a new version of this document.

Original Authors: Steven Songwe and Enock Chizuzu **This Revision:** Enock Chizuzu, with Claude (PM/Advisor)